

RESUMO EXECUTIVO E RELATÓRIO FINAL - SEAZONE DATA CHALLENGE

AI Builder Senior - Projetos Concluídos

▢ PARTE 1: INGESTÃO DE DADOS & MODELAGEM (main.py)

Objetivo

Ler 5 CSVs da pasta `data/`, identificar reservas reais através da diferença de snapshots, calcular receita e fazer joins com dados de detalhes.

Stack

- **Python 3** com `uv` package manager
- **DuckDB** para SQL local (ingestão e aggregations)
- **Polars** para transformações de dados (sem Pandas/Jupyter)

Resultados

- ☐ Schema descoberto dinamicamente (leitura de headers)
- ☐ Ingestão com `pathlib` (caminhos dinâmicos)
- ☐ CTE DuckDB corrigida: **228 imóveis com diárias reservadas**
- ☐ Output gerado: `analysis/revenue_analysis.csv`

Dados Consolidados:

- 228 imóveis alugados
- **Colunas:** `airbnb_listing_id`, `diarias_alugadas`, `receita_gerada`, `number_of_bedrooms`, `bairro`

▢ PARTE 1: DISCOVERY & EXPLORAÇÃO (discovery.py)

Objetivo

Responder às 2 primeiras perguntas de negócio usando aggregations no Polars.

Resultados

Pergunta 1: Melhor Localização (Top 5 Bairros) | Bairro | Receita Total | Receita Media/Imovel | Qtd | | ----- | ----- | ----
-- | ----- | | **Meia Praia** | R\$ 3.350.800 | R\$ 25.579 | 131 | | Centro | R\$ 607.076 | R\$ 12.142 | 50 | | Morretes | R\$ 365.067
| R\$ 16.594 | 22 | | Casa Branca | R\$ 135.368 | R\$ 16.921 | 8 | | Sem Bairro | R\$ 124.525 | R\$ 41.508 | 3 |

Insight: Meia Praia = 82% da receita com 57% dos imóveis.

Pergunta 2: Melhor Perfil de Imóvel | Quartos | Receita Total | Receita Media | Media Diarias | | ----- | ----- | ----- | -----
| | **3** | **R\$ 1.712.552** | R\$ 20.633 | 25,3 | | **4** | **R\$ 1.387.120** | **R\$ 81.595** | **36,6** | | **2** | R\$ 1.145.589 | R\$ 15.074 | 28,2 | | **1**
| R\$ 434.320 | R\$ 9.652 | 25,8 |

Insight: Imóveis de 4 quartos = 4x maior receita média (melhor ROI).

▢ **PARTE 1: MODELAGEM FINANCEIRA & ROI (roi_calculator.py)**

Objetivo

Projetar viabilidade de um prédio de 50 apartamentos (4 quartos) em Meia Praia para 2025-2027.

Premissas

- Base: R\$ 81.595 de receita média por imóvel/ano (discovery)
- Crescimento: 5% ao ano (inflação/valorização)
- Custos: 35% da receita (limpeza, manutenção, taxa, gestão)
- Margem: 65% de lucro operacional

Projeções

Metrica	2025	2026	2027
Receita Bruta	R\$ 4.079.750	R\$ 4.283.738	R\$ 4.497.924
Custos (35%)	R\$ 1.427.913	R\$ 1.499.308	R\$ 1.574.274
Lucro	R\$ 2.651.838	R\$ 2.784.429	R\$ 2.923.651

Resumo 3 Anos: Lucro Total: **R\$ 8.359.918** | ▢ Projeto VIÁVEL com 65% de margem.

▢ **PARTE 2: PRODUTO INTELIGÊNCIA BRASIL (Specs)**

Objetivo: Especificar a arquitetura de escala nacional do produto usando metodologia Spec-Driven Development.

Resultados:

- ▢ **constitution.md:** Regras inegociáveis do squad (Idempotência, custo-bound em BigQuery/Athena).
- ▢ **spec.md:** Definição do produto usando granularidade espacial H3 para cruzamento perfeito de demanda litorânea.
- ▢ **plan.md:** Divisão estratégica da execução em 3 fases entre os squads Data Core, Acquisition e Edge.

▢ **PARTE 4: LIDERANÇA TÉCNICA E PLANO DE SQUAD**

Objetivo: Estruturar a condução do time de dados para os primeiros 30/60/90 dias e resolver underperformance.

Resultados:

- ☐ **Plano 30/60/90:** Diagnóstico focado em adoção real de AI Builder, implementação de "Spec Review" pré-código e métricas de infraestrutura.
 - ☐ **Gestão de Underperformance:** Plano de ação direto para realinhar um desenvolvedor Sênior ao novo paradigma de agentes autônomos, com gatilhos claros de offboarding (D+60) caso não haja adaptação metodológica, visando proteger o *throughput* do squad.
-

☐ AI CO-AUTHOR LOG (Sessões de IA)

- **O que entreguei à IA:** Arquivos de especificação (`spec.md` e `plan.md`) para pautar o contexto, seguidos de prompts guiados instruindo restrições rígidas de stack (uso de `uv`, DuckDB e Polars).
 - **Onde a IA acelerou bem:** Geração do boilerplate (setup do ambiente com `uv`) e escrita rápida das agregações no Polars para as perguntas de negócio (Discovery). Ganho de speedup estimado em 60%.
 - **Onde a IA errou ou desviou e como corrigi:** Na primeira tentativa de modelagem no DuckDB, a IA errou a lógica matemática do "sumiço" das diárias entre as datas de aquisição (trouxo apenas 1 linha em vez das 228 reais). Corrigi enviando um novo prompt forçando a reescrita do SQL para identificar a intersecção correta das datas de snapshot.
 - **Logs Gerados:** `00-setup-agentes.md`, `01-discovery.md`, `02-modelagem.md`, `03-roi.md`.
-

☐ ESCOLHAS DE STACK JUSTIFICADAS

- **uv:** Utilizado para garantir um gerenciamento rápido e determinístico de dependências, substituindo o uso de `requirements.txt` solto.
 - **DuckDB:** Escolhido como engine SQL local simulando perfeitamente a arquitetura, os custos e a modelagem do AWS Athena / GCP BigQuery em produção.
 - **Polars:** Adotado para a análise exploratória (wrangling), garantindo performance superior ao Pandas e evitando vazamento de memória.
-

☐ FEEDBACK SOBRE O DESAFIO E MELHORIAS FUTURAS

Excelente desafio que testou não apenas a capacidade técnica de codificação, mas a verdadeira capacidade de liderança arquitetural e *spec-driven development*. Como melhoria futura para o produto "Inteligência Brasil", podemos implementar um Agente de IA autônomo atuando diretamente no pipeline de CI/CD para rodar testes de idempotência e dry-runs de custo nas queries antes mesmo do código chegar a um revisor humano.

☐ STATUS FINAL DA ENTREGA

- **Data:** 26/05/2026
- **Repositório GitHub:** Atualizado e configurado com AI Builder specs.
- **Metodologia Comprovada:** Histórico do Git (`git log --reverse`) atesta especificações antecedendo o código.